Review of Relational Databases

Overview

- **Describe the relational database model**
- **Explain basic SQL functions**
- **Recognize the OMOP database**

Async

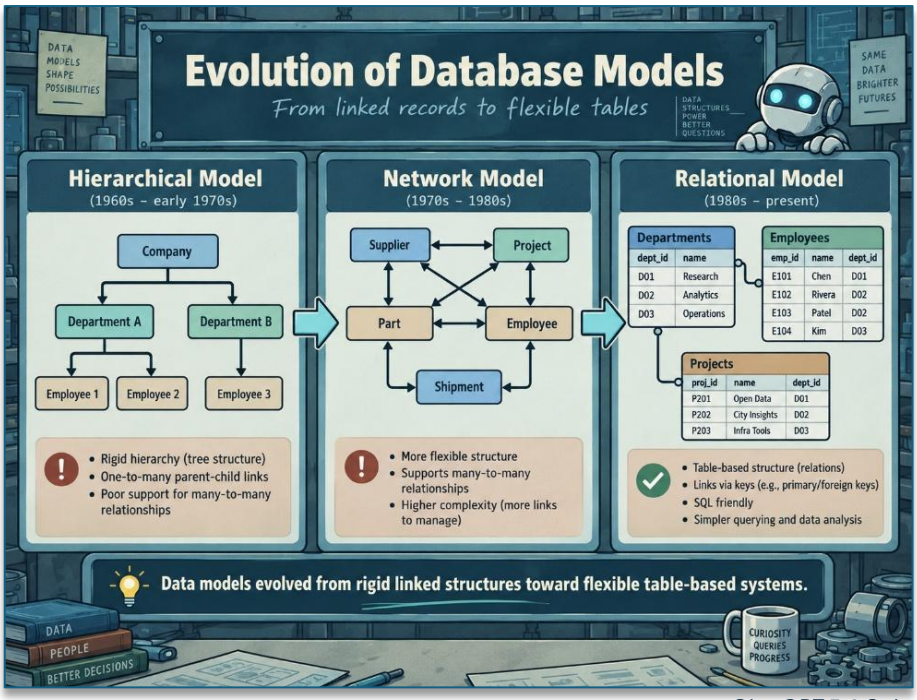
The Relational Model

Data Models	• a collection of concepts used to describe: ○ structure of a database ○ relationships between data elements • the history of databases is closely tied to the development of different data models ○ conceptual data model ▪ high-level ▪ describes how users understand relationships between data ▪ main focus of this course ○ physical data model ▪ more technical ▪ describes how data is physically stored on computer hardware ▪ *not a major focus of this course*

Hierarchical Model – 1960s

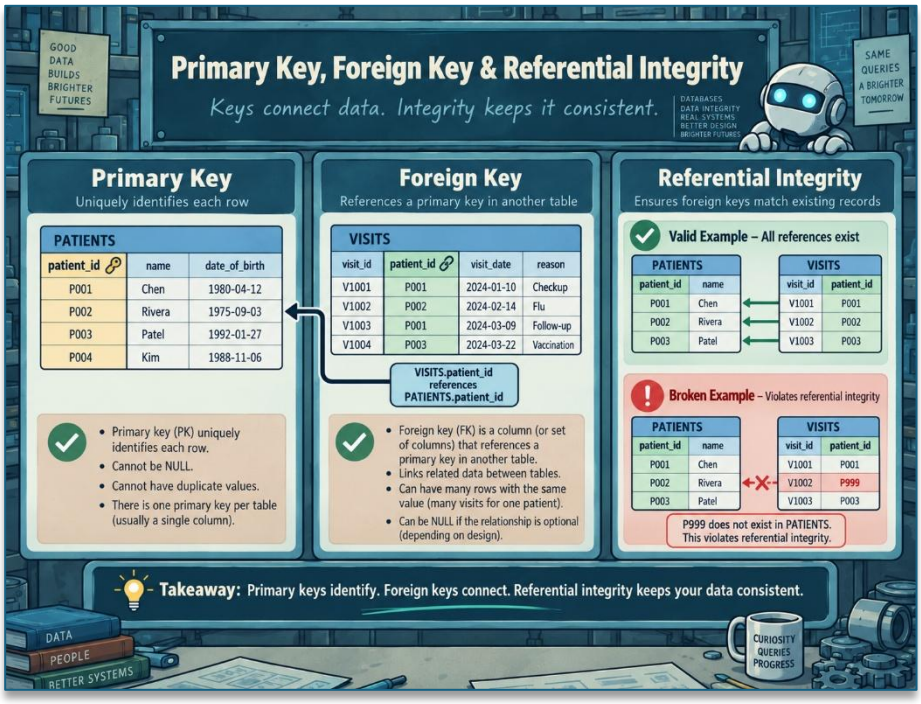
Organizes data in a top-down tree structure.

- primarily supports one-to-many relationships
  - a department
    - can have many employees
  - an employee
    - belongs to only one department
    - can have many projects
    - can have many computers
  - a project
    - belongs to only one employee
    - a computer
    - belongs to only one person



ChatGPT 5.6 Sol

	○ **key rule** ▪ every child can only have one parent ○ **query limitation** ▪ must begin at the top of the hierarchy
Network Model – 1960s	**The network model expanded the hierarchical model.** • its major improvement was support for many-to-many relationships • advantages ○ more flexible than the hierarchical model • problems ○ difficult to maintain ○ structural changes can be complicated ○ not every real-world relationship fits naturally into a parent-child structure • a more flexible model is still needed
Relational Model – 1970s	**The relational model was developed primarily through the work of E. F. Codd.** • it became the foundation of modern relational databases • a major improvement was separating physical data storage from conceptual presentation of data • this makes databases easier to: ○ query ○ update ○ manage



ChatGPT 5.6 Sol

Properties of Relational Databases

Data Is stored in tables of rows and columns.

- primary keys (or *unique key*)
  - a unique identifier for a row in a table
- relations have no inherent order
- columns have no inherent order
- attributes must be atomic (can only contain one value)
- NULL values
  - literally "nothing" – not "zero"
  - NULL ≠ NULL

Relational Model: Constraints

Constraints

A rule used when designing and working with a relational database.

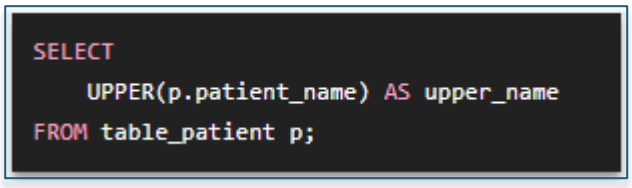
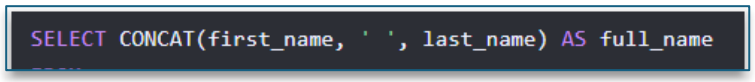
- help control:

	○ what data can be stored ○ which values are valid ○ how records are uniquely identified ○ how different tables may reference one another
Three Types of Constraints	• **implicit** ○ built into the relational database system ○ the user does not normally choose or manage these • **explicit** ○ rules deliberately defined by the database designer • **business rules** ○ usually more complex rules associated with the application using the database ○ instead of allowing invalid data to reach the database, the application layer (ex: a web form) can enforce these rules first
Domain Constraints	**Defines what kind of data can appear in an attribute or column.** • commonly enforced using data types that are normally defined when the table is created • values stored in that column must follow the defined domain
Key Constraints	**Primary key uniquely identifies one row in a relation.** • each value must be unique
Key Design Must Account for Future Data	**A key may work with the database's current data but fail when new records are added late.** • therefore, database designers should consider

	○ existing records ○ expected future records ○ whether the chosen key will remain unique • a poor key design can prevent legitimate future data from being stored
Composite Keys	**Uses multiple attributes together to uniquely identify a row.**
Database Schema	**A map of the database.** • every relation has a schema, and the schemas of all relations together describe the overall database structure • looking at the complete schema helps designers make decisions based on the **entire database**, rather than designing each table independently
Entity Integrity	**An entity integrity constraint is an implicit database constraint.** ○ example: a primary key can never be null
Referential Integrity	**Referential integrity controls relationships between different relations.** • example: ○ if a record in Table B refers to a record in Table A, the referenced record must actually exist in Table A
Foreign Keys	**A foreign key :** • is an attribute that references the primary key of another relation ○ does not have to be the primary key of its own table ○ only needs to reference the appropriate primary key in another table

	• foreign keys are essential because they allow relational databases to: ○ connect tables ○ query information across tables ○ represent relationships between entities
SQL Basics	
FROM, SELECT, and WHERE	**SQL queries are built from three fundamental clauses.** • required clauses: ○ FROM ▪ which table(s) provide the data ○ SELECT ▪ which columns appear in the output • optional clause: ○ WHERE ▪ which rows are returned `SELECT what you want to see` `FROM where the data comes from` `WHERE which records you want`
FROM: Choose the Table	**Tells the database which table contains the data.** • table aliases ○ a table can be given a shorter temporary name called an alias `FROM table_patient p` ○ now '***p***' represents '`table_patient`' for the rest of the query

SELECT: Choose the Columns	**Determines which columns appear in the query output.** ``` SELECT p.patient_name, p.patient_dob, p.patient_race FROM table_patient p; ``` ○ the table may contain many more columns, but only these three will appear • qualifying column names ○ with its table name ``` table_patient.patient_name ``` ○ with the table's alias ``` p.patient_name ```
SQL Functions in SELECT	**Transform or calculate values before they appear in the output.** ``` SELECT UPPER(p.patient_name) FROM table_patient p; ``` ○ UPPER() converts to uppercase ○ so 'Joe Waller' becomes 'JOE WALLER'
Renaming Output Columns	**A calculated or transformed column can be given an alias using AS.**

	 ○ does not rename the column in the original database ○ only changes the name shown in the query output
Combining Column Values	**SQL functions can also combine values from multiple columns.** 
Core Functions	**String Functions.** • UPPER() ○ converts text to uppercase • LOWER() ○ converts text to lowercase • concatenation ○ combines two or more string values into a single value **Aggregate Functions.** • COUNT() ○ counts the number of returned records or values • AVG() ○ calculates the average of numeric values **Mathematical Functions.** • FLOOR() ○ rounds a number down • CEILING() ○ rounds a number up

	• Standard mathematical functions ○ perform common arithmetic calculations **Date Functions.** • date-difference functions ○ calculate the amount of time between dates • age/date calculations ○ calculate values such as a person's age from stored dates • Notes ○ functions can be used in both SELECT and WHERE clauses ○ exact function syntax can vary between relational database management systems ○ this course uses PostgreSQL (Postgres) syntax
Data Manipulation	
Manipulating Tables and Data with SQL	**SQL can do more than retrieve data.** • it can also: ○ create and modify tables ○ insert new rows ○ update existing values ○ delete selected rows ○ remove all rows ○ remove entire tables • these operations can permanently alter a database ○ for this reason: ▪ database administrators usually restrict them to authorized users

Two Types of Modification

Target	Purpose	Main commands
Database structure	Change tables, columns, and constraints	CREATE, ALTER, DROP
Stored data	Add, change, or remove rows	INSERT, UPDATE, DELETE, TRUNCATE

The distinction between structure and data is important:

- ALTER
  - changes the table's definition
- UPDATE
  - changes values stored inside the table
- DROP
  - removes the table itself
- TRUNCATE
  - removes its rows but preserves the table

Creating a Table

The CREATE TABLE statement defines a new table.

- a table definition normally specifies:
  - the table name
  - column names
  - data types
  - nullability rules
  - keys and other constraints

```
CREATE TABLE PATIENT_DIMENSION (
    PATIENT_NUM     INT          NOT NULL,
    PATIENT_NAME    VARCHAR(100) NOT NULL,
    RELIGION_CODE   VARCHAR(20),
    CREATED_AT      TIMESTAMP,

    CONSTRAINT PATIENT_DIMENSION_PK
        PRIMARY KEY (PATIENT_NUM)
);
```

Column Names

Each column must have a name.

- a consistent naming convention improves:
  - readability
  - maintainability

Common conventions include:

- using one consistent letter case
- separating words with underscores
- giving related columns similar names
- avoiding ambiguous abbreviations
- following the same convention across all tables

The specific convention matters less than applying it consistently.

Data Types

- A column's data type controls which values it can store.

Data type	Typical purpose
INT	Whole numbers
VARCHAR(n)	Variable-length text
DATE	Calendar dates
TIMESTAMP	Dates and times
DECIMAL(p,s)	Fixed-precision numeric values
BOOLEAN	True-or-false values

The exact data types and syntax vary among database systems.

- choosing appropriate data types improves:
  - data validation
  - storage efficiency
  - query performance

	○ consistency **Compatibility with database functions.** •
Null Values	**A NULL value represents missing, unknown, or unavailable information.** • by default, many columns permit null values • the NOT NULL constraint requires a value `PATIENT_NUM INT NOT NULL` • whether a column should permit NULL depends on the meaning of the data ○ for example: ○ a patient identifier may always be required ○ a patient's religion may be unknown or intentionally unreported • a column omitted from an INSERT statement must either: ○ permit NULL, or ○ have a default value ○ otherwise, the insertion will fail
Primary-Key Constraints	**A primary key uniquely identifies each row.** `CONSTRAINT PATIENT_DIMENSION_PK PRIMARY KEY (PATIENT_NUM)` ○ this definition includes: ▪ a constraint name:

<table>
<tr><td></td><td>

PATIENT_DIMENSION_PK

a constraint type:

PRIMARY KEY

a key column:

PATIENT_NUM

naming constraints is useful because a constraint may later need to be:

modified
removed
referenced in an error message
examined during maintenance

a primary key normally enforces:

uniqueness
non-null values

</td></tr>
<tr><td>Altering a Table</td><td>The ALTER TABLE statement changes an existing table’s structure.

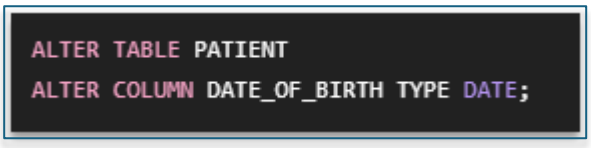
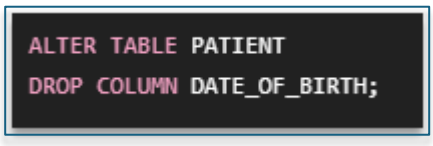
it does not modify values stored in existing rows unless the structural change causes a database-specific conversion

</td></tr>
<tr><td>Adding a Column</td><td>This adds DATE_OF_BIRTH to the table.
<pre><code>ALTER TABLE PATIENT
ADD DATE_OF_BIRTH TIMESTAMP;</code></pre>

existing rows will generally receive NULL for the new column unless:

a default value is supplied, or
the database applies another defined rule

</td></tr>
</table>

	Adding a NOT NULL column to a populated table may require a default value or a staged migration.
Changing a Column	**This changes the column's data type.**  • the exact syntax differs across database systems • a type change may fail if existing values cannot be converted safely • before changing a type, consider whether: ○ current values are compatible ○ precision could be lost ○ applications depend on the current type ○ indexes or constraints use the column ○ a backup or rollback plan exists
Removing a Column	**This removes the column and its stored values.**  • dropping a column may also affect: ○ constraints ○ indexes ○ views ○ stored procedures ○ reports

```
ALTER TABLE PATIENT
ALTER COLUMN DATE_OF_BIRTH TYPE DATE;
```

```
ALTER TABLE PATIENT
DROP COLUMN DATE_OF_BIRTH;
```

	○ application code **Dependencies should be identified before the change is applied.**
Dropping a Table	**The DROP TABLE statement removes an entire table.** ```DROP TABLE PATIENT;``` • **This may remove:** ○ **all rows** ○ **column definitions** ○ **constraints** ○ **indexes** ○ **associated metadata** • **recovery depends on:** ○ **database system** ○ **transaction configuration** ○ **backups** ○ **administrative policies** **DROP TABLE should therefore be treated as a potentially destructive operation.**
SQL Injection Connection	**The well-known XKCD "Bobby Tables" comic uses a malicious name containing a command similar to:** ```DROP TABLE STUDENTS;``` • if an application directly combines user input with SQL text

<table>
<tr>
<td></td>
<td>

the input may be interpreted as executable SQL

the correct defense is not merely removing suspicious words

applications should use:

parameterized queries
prepared statements
input validation
least-privilege database accounts
appropriate error handling

User input should never be inserted directly into executable SQL strings.
</td>
</tr>
<tr>
<td>Inserting Data</td>
<td>
The INSERT statement adds new rows.
<pre>
INSERT INTO TABLE_PATIENT (
    PATIENT_ID,
    DATE_OF_BIRTH,
    SSN,
    HOME_PHONE
)
VALUES (
    1001,
    '1985-04-12',
    '000-00-0000',
    '555-0100'
);
</pre>

the listed values correspond to the columns in the same order
an insertion must satisfy the table’s rules:

column names must exist
values must use compatible data types
required columns must receive values
primary-key values must be unique
foreign-key relationships must be valid

</td>
</tr>
</table>

	○ check constraints must be satisfied **It is good practice to list column names explicitly. This makes the statement clearer and less dependent on the physical order of columns.**
Deleting Rows	**The DELETE statement removes rows while preserving the table's structure.** ``` DELETE FROM PATIENT_VISIT WHERE EXTRACT(YEAR FROM DATE_OF_BIRTH) = 1942; ``` • only rows satisfying the WHERE condition are removed • function names and date syntax differ across database platforms ○ some systems use: ``` YEAR(DATE_OF_BIRTH) ```
Importance of the WHERE Clause	**A DELETE statement without a WHERE clause removes every row:** ``` DELETE FROM PATIENT_VISIT; ``` • a filtered deletion is usually safer: ``` DELETE FROM PATIENT_VISIT WHERE PATIENT_ID = 1001; ``` • before executing a destructive statement, the condition can be tested with SELECT:

<table>
<tr><td></td><td>

```sql
SELECT *
FROM PATIENT_VISIT
WHERE PATIENT_ID = 1001;
```

This confirms which rows will be affected.

</td></tr>
<tr><td>Updating Existing Data</td><td>

The UPDATE statement changes stored values.

```sql
UPDATE PATIENT_VISIT
SET PROVIDER_NAME = 'ELKIN'
WHERE PROVIDER_ID = 626;
```

- this changes PROVIDER_NAME only for rows matching the condition

</td></tr>
<tr><td>Selecting Rows Precisely</td><td>

Updates should use conditions that identify the intended rows as precisely as possible.

- preferred:

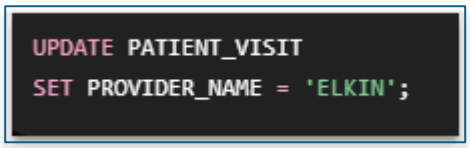
```sql
UPDATE PATIENT_VISIT
SET PROVIDER_NAME = 'ELKIN'
WHERE PROVIDER_ID = 626;
```

- potentially unsafe:

```sql
UPDATE PATIENT_VISIT
SET PROVIDER_NAME = 'ELKIN'
WHERE PROVIDER_NAME = 'HANSEN';
```

- names are not necessarily unique
  - the second statement could modify multiple unrelated providers who share the same surname

</td></tr>
</table>

	Primary keys or other unique identifiers are generally safer for targeted changes.
Updating Without a Filter	**An UPDATE statement without a WHERE clause modifies every row:**  `UPDATE PATIENT_VISIT` `SET PROVIDER_NAME = 'ELKIN';` • this may be intentional, but it is often a serious error • before updating data: ○ write the intended WHERE condition ○ test it with SELECT ○ confirm the expected row count ○ execute the update within a transaction when supported ○ validate the result ○ commit only after verification
Truncating a Table	**The TRUNCATE TABLE statement removes all rows while preserving the table definition.**  `TRUNCATE TABLE PATIENT_VISIT;` • the table remains available, but it becomes empty • depending on the database system, truncation may also reset: ○ identity counters ○ auto-increment sequences ○ storage allocations

DELETE, TRUNCATE, and DROP

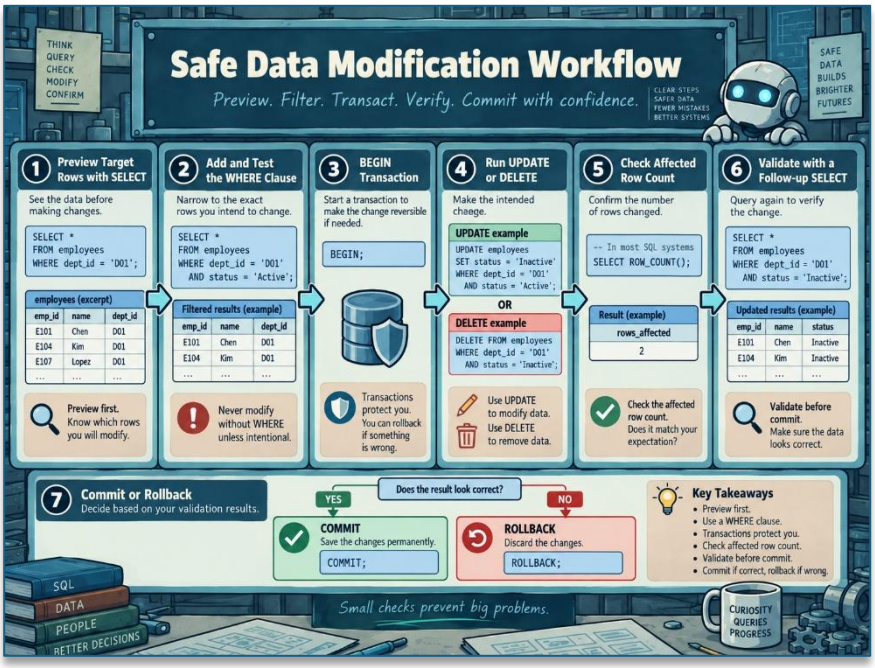
Command	Removes selected rows	Removes all rows	Preserves table structure
DELETE ... WHERE	Yes	Only if all rows match	Yes
DELETE without WHERE	No selective filter	Yes	Yes
TRUNCATE TABLE	No	Yes	Yes
DROP TABLE	Not applicable	Yes	No

- DELETE
  - can remove selected rows
  - supports a WHERE clause
  - usually records row-level changes
  - may trigger delete-related database logic
  - can be slower when removing every row from a large table
- TRUNCATE
  - removes all rows
  - does not support row filtering
  - is generally faster than deleting rows individually
  - usually performs less row-level logging
  - may have stricter permission and foreign-key restrictions
- DROP
  - removes the complete table
  - removes its structure and stored data
  - may break dependent database objects and applications

Transaction and recovery behavior varies by database system.

TRUNCATE and DROP should not be assumed to be reversible.

Safe Data-Modification Practices



ChatGPT 5.6 Sol

Before running UPDATE, DELETE, TRUNCATE, ALTER, or DROP:

- confirm that the correct database is selected.
- confirm the schema and table name.
- inspect the affected rows with SELECT.
- use primary keys or unique identifiers where possible.
- check the expected number of affected rows.
- identify dependent applications and database objects.
- use transactions when the database supports them.
- maintain current backups.
- follow change-review and approval procedures.
- grant users only the permissions required for their roles.

A typical controlled modification uses a transaction:

```
BEGIN;

UPDATE PATIENT_VISIT
SET PROVIDER_NAME = 'ELKIN'
WHERE PROVIDER_ID = 626;

-- Validate the result before committing.

COMMIT;
```

- if validation fails:

```
ROLLBACK;
```

Transaction syntax and support depend on the database platform and operation.

Command Summary

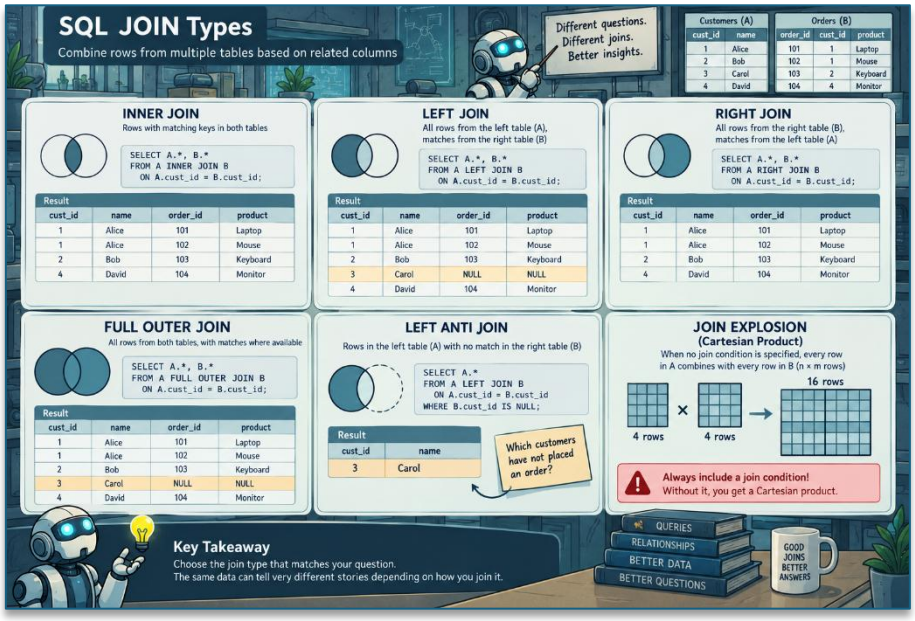
Command	Purpose
CREATE TABLE	Create a new table
ALTER TABLE	Change a table's structure
DROP TABLE	Remove a table and its contents
INSERT	Add new rows
UPDATE	Change values in existing rows
DELETE	Remove selected or all rows
TRUNCATE TABLE	Efficiently remove all rows while preserving the table

JOINs

Joining Tables with SQL

A SQL join combines related data from two or more tables.

- joins are fundamental to relational databases because normalized data is usually distributed across multiple tables
  - for example:
    - a PATIENT table stores patient information
    - a VISIT table stores clinical visits
    - a PROVIDER table stores provider information



ChatGPT 5.6 Sol

A join can combine these tables into one query result.

Joins and Set Theory

Joins are often introduced using set intersections.

- suppose one set contains mammals and another contains pets
  - the animals that belong to both sets might include:
    - dogs
    - cats
    - guinea pigs

- the overlap resembles an inner join because only matching members are retained
  - however, SQL joins operate on rows and matching conditions
  - they are not exactly equivalent to mathematical set operations because:
    - SQL tables may contain duplicate rows
    - one row may match multiple rows

	▪ SQL uses NULL to represent missing values ▪ join cardinality depends on key relationships
Join Keys	**Tables are usually connected through primary-key and foreign-key relationships.** `PATIENT.PATIENT_ID`
Foreign Key	A foreign key references a key in another table. `VISIT.PATIENT_ID` • the relationship can be expressed as: `PATIENT.PATIENT_ID = VISIT.PATIENT_ID` • database documentation is important because it identifies: ○ primary keys ○ foreign keys ○ table relationships ○ expected relationship cardinalities
Table Aliases	**Aliases assign short names to tables within a query.** `FROM PATIENT AS P` • the alias P can then qualify columns: `P.PATIENT_NAME` • aliases are especially useful when: ○ several tables contain columns with the same name

<table>
<tr><td></td><td>

queries contain many joins
a table is joined to itself
fully qualified table names are long
example:


```
SELECT
    P.PATIENT_NAME,
    V.VISIT_DATE
FROM PATIENT AS P
INNER JOIN VISIT AS V
    ON P.PATIENT_ID = V.PATIENT_ID;
```

</td></tr>
<tr><td>Two Ways to Write Joins</td><td>
SQL supports an older implicit syntax and a modern explicit syntax.

Implicit Join Syntax

the older form lists tables in the FROM clause and places the join condition in WHERE


```
SELECT
    P.PATIENT_NAME,
    V.VISIT_DATE
FROM PATIENT AS P,
     VISIT AS V
WHERE P.PATIENT_ID = V.PATIENT_ID;
```


this form separates:

ON:

how tables are related

WHERE:

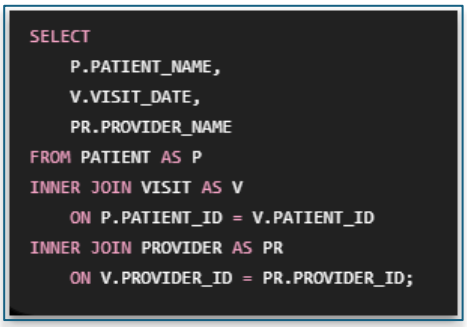
which joined rows should remain

explicit joins generally improve:

readability
maintainability
error detection
support for outer joins

</td></tr>
</table>

<table>
<tr><td>Joining Multiple Tables</td><td>Additional tables can be connected by adding more join clauses.

• this query connects:
○ patients to visits through PATIENT_ID
○ visits to providers through PROVIDER_ID</td></tr>
<tr><td>The $(n-1)$ Heuristic</td><td>For a simple connected chain of (n) tables, at least $(n-1)$ join relationships are normally required.

Tables — Minimum connecting relationships
2 — 1
3 — 2
4 — 3

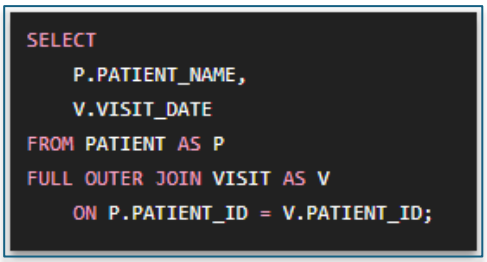
• this is a useful check, but not a universal SQL rule
• a query may require additional conditions when:
○ tables use composite keys
○ several relationships exist between the same tables
○ date ranges or version columns participate in the join
○ the join graph contains multiple valid paths

The actual requirement is that every table be connected correctly according to the data model.</td></tr>
<tr><td>Join Cardinality</td><td>The number of output rows depends on how many matches exist.</td></tr>
</table>

	• one-to-one ○ each row matches no more than one row in the other table • one-to-many ○ one row may match several rows ○ for example: ▪ one patient may have several visits ▪ the patient's name will appear once for every matching visit • many-to-many ○ several rows in each table may match several rows in the other table ○ this can produce a much larger result than either source table ○ many-to-many relationships are often represented through a junction table • understanding cardinality helps distinguish: ○ expected duplicate-looking rows ○ incorrect join conditions ○ unexpected row multiplication
Inner Join	**An INNER JOIN retains only rows that satisfy the join condition.** `SELECT` `  P.PATIENT_NAME,` `  V.VISIT_DATE` `FROM PATIENT AS P` `INNER JOIN VISIT AS V` `  ON P.PATIENT_ID = V.PATIENT_ID;`

	• the result includes: ○ patients with matching visits ○ one row for each matching patient-visit combination • the result excludes: ○ patients without visits ○ visits without matching patients **For a simple equality condition, reversing the order of the two tables does not normally change which matched combinations appear.**
Left Join	**A LEFT JOIN, also called a left outer join, retains:** • every row from the left table • matching rows from the right table • NULL values where no right-table match exists ```sql SELECT P.PATIENT_NAME, V.VISIT_DATE FROM PATIENT AS P LEFT JOIN VISIT AS V ON P.PATIENT_ID = V.PATIENT_ID; ``` • this query returns every patient ○ patients without visits remain in the result ○ but their visit fields are NULL • a left join is useful for questions such as: ○ which patients have no visits? ○ which customers have never placed an order? ○ which employees have no assigned supervisor?

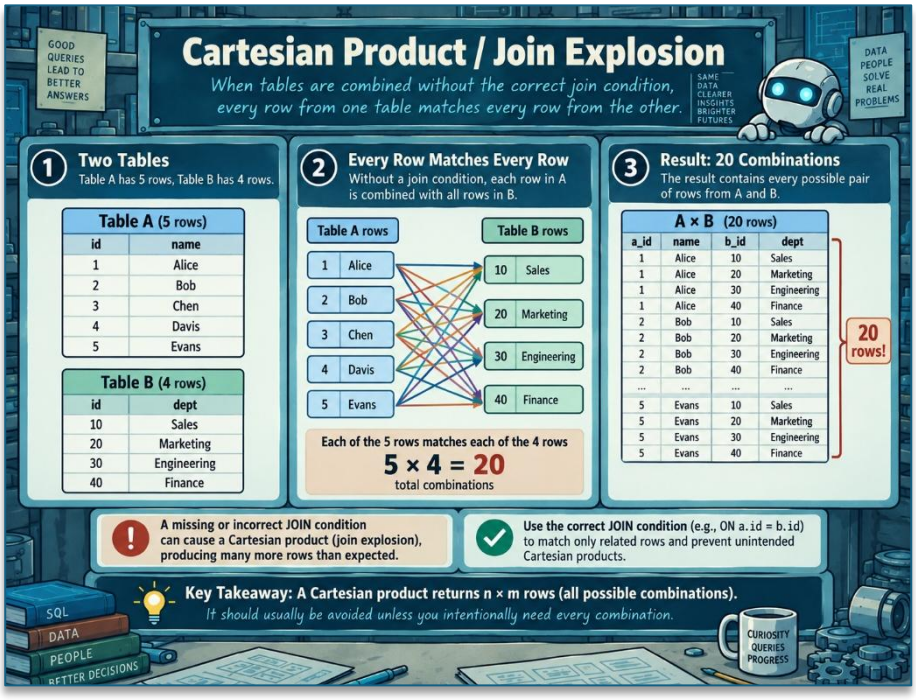
	○ which products have no sales?
Finding Unmatched Rows	A left join can identify rows without a corresponding match. ```sql SELECT P.PATIENT_ID, P.PATIENT_NAME FROM PATIENT AS P LEFT JOIN VISIT AS V ON P.PATIENT_ID = V.PATIENT_ID WHERE V.PATIENT_ID IS NULL; ``` • this pattern is called an anti-join • it returns patients who have no matching visit
Right Join	**A RIGHT JOIN, or right outer join, retains every row from the right table.** ```sql SELECT P.PATIENT_NAME, V.VISIT_DATE FROM VISIT AS V RIGHT JOIN PATIENT AS P ON V.PATIENT_ID = P.PATIENT_ID; ``` • This can produce the same result as: ```sql SELECT P.PATIENT_NAME, V.VISIT_DATE FROM PATIENT AS P LEFT JOIN VISIT AS V ON P.PATIENT_ID = V.PATIENT_ID; ``` • many developers primarily use left joins and reorder the tables when necessary ○ this creates a consistent reading rule: ▪ preserve every row from the table written on the left

	Right joins are valid, but not every database system supports them.
Full Outer Join	**A FULL OUTER JOIN preserves:** • every row from the left table • every row from the right table • combined rows where the join condition matches • NULL values on the missing side of unmatched rows  • this result contains: ○ patients with visits ○ patients without visits ○ visits without matching patients • full outer joins are useful for: ○ comparing datasets ○ reconciling records ○ detecting missing relationships ○ identifying additions and removals between data versions **Some database systems do not directly support FULL OUTER JOIN.**

```
SELECT
    P.PATIENT_NAME,
    V.VISIT_DATE
FROM PATIENT AS P
FULL OUTER JOIN VISIT AS V
    ON P.PATIENT_ID = V.PATIENT_ID;
```

Cartesian Product

A Cartesian product combines every row from one table with every row from another table.



ChatGPT 5.6 Sol

- **the explicit syntax is:**

```
SELECT *
FROM PATIENT
CROSS JOIN LAB_RESULT;
```

- **If the first table has (n) rows and the second has (m) rows, the result contains $n \times m$ rows**
  - **for example:**

$$5,000,000 \times 45,000,00 = 225,000,000,000,000$$

    - **that is 225 trillion row combinations!**

A Cartesian product can occur accidentally when using implicit join syntax without a join condition:

<table>
<tr>
<td></td>
<td>

```
SELECT *
FROM PATIENT AS P,
     VISIT AS V;
```

- it can also occur when a table is added to a query but never connected to the other tables

- possible consequences include:
  - extremely large result sets
  - long-running queries
  - excessive memory consumption
  - high CPU and storage use
  - database instability
  - unexpected cloud costs

- across join is not inherently incorrect
- it is useful when every possible combination is intentionally required.

The danger is producing one unintentionally.

</td>
</tr>
<tr>
<td>Self-Joins</td>
<td>

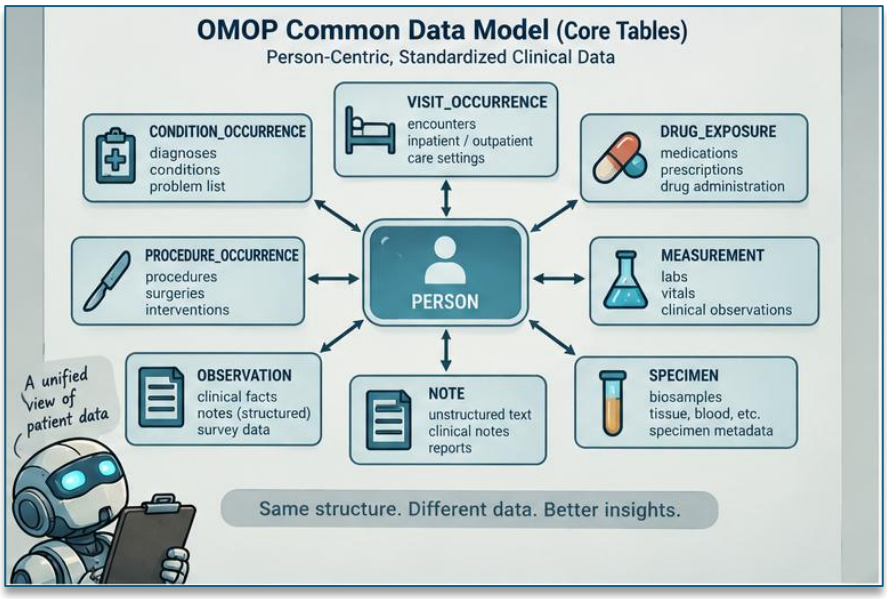
A self-join joins a table to another logical instance of itself.

- this is useful when rows in a table refer to other rows in the same table
- consider an employee table:

Column	Meaning
EMPLOYEE_ID	Unique employee identifier
EMPLOYEE_NAME	Employee's name
MANAGER_ID	Identifier of the employee's manager

</td>
</tr>
</table>

	• managers are also employees ○ so both employees and managers appear in the same table
Introduction to OMOP	
What Is OMOP?	**OMOP is a healthcare Common Data Model (CDM).** • a common data model defines a shared way for different organizations to structure their data ○ UNC and Duke can store healthcare data using OMOP ○ researchers at both institutions can then use similar queries and analytical code **Main Benefit.** • different healthcare institutions can represent similar information in a standardized structure • this makes: ○ data sharing easier ○ multi-institution research easier ○ analytical code more reusable ○ and healthcare data more consistent
Understanding the Database Diagram	**A database diagram shows:** • which tables exist • how those tables connect **Some diagrams also show:** • attributes • primary keys

	• foreign keys • detailed table relationships **The OMOP diagram introduced here is intentionally high-level.**
Core OMOP Structure	**Two tables form the foundation of much of the model:**  *ChatGPT 5.6 Sol* **PERSON.** • stores information about patients • each patient should have: ○ one unique record ○ identified by a unique person identifier • the person table can also connect to information about: ○ locations ○ healthcare providers ○ care sites

VISIT_OCCURRENCE.

- the VISIT_OCCURRENCE table stores information about healthcare visits or encounters
    - examples include:
        - emergency department visits
        - hospital admissions
        - outpatient appointments
        - eye examinations
        - other encounters with the healthcare system

- One patient can have many visits
    - therefore:
        - person_id may appear many times
        - while visit_occurrence_id uniquely identifies each visit
    - each visit can produce many additional healthcare records

CONDITION_OCCURRENCE.

- the CONDITION_OCCURRENCE table stores diagnoses and medical conditions
    - examples include:
        - diseases
        - medical conditions
        - diagnoses associated with routine well visits
    - a patient can have many diagnoses
    - a single visit can also contain multiple diagnoses
        - therefore:
            - diagnoses are not unique by patient or by visit

DRUG_EXPOSURE.

- the DRUG_EXPOSURE table stores information about medications
    - it may contain:
        - prescriptions written during care
        - medications currently being taken
        - medication history reported by the patient

- a single medication may have many different codes
    - this can happen because drugs differ by:
        - formulation
        - dosage
        - combination
        - product

PROCEDURE_OCCURRENCE.

- the PROCEDURE_OCCURRENCE table stores medical procedures
- procedures are not limited to surgery
    - they may include:
        - surgery
        - injections
        - infusions
        - ultrasounds
        - other clinical procedures
- different versions of the same general procedure can receive different standardized identifiers

MEASUREMENT.

- the MEASUREMENT table stores clinical measurements
    - examples include:

- blood pressure
- laboratory results
- body temperature
- weight
- height

- measurements usually require three pieces of information:
  - what was measured
  - the measured value
  - the unit
- for example:
  - measurement: body temperature
  - value: 103
  - unit: degrees Fahrenheit

- knowing only that a temperature was measured is not enough
  - the actual value and unit are needed for analysis

OBSERVATION.

- the OBSERVATION table stores information that does not fit naturally into the other OMOP tables
- the lecture describes it as something of a “junk bucket”
- examples may include:
  - medical history
  - family history
  - social factors
  - numerical values
  - text strings

	The table is intentionally flexible because it may need to store many unrelated types of information.
OMOP Uses Codes Instead of Human Readable Text	**A major feature of OMOP is its heavy use of coded values and identifiers.** • instead of storing: ○ female • the database might store: ○ 8532 • instead of storing: ○ endometriosis • the database might store ○ a numerical concept ID **The meaning of these codes is retrieved through lookup tables and standardized vocabularies.** • this is common in well-structured databases • core tables often contain compact identifiers rather than long human-readable labels
Why Use Codes?	**Healthcare contains enormous numbers of:** • diagnoses • medications • procedures • measurement types • variations of each **Using standardized identifiers helps keep the database:**

	• organized • consistent • searchable • interoperable **Lookup tables provide the human-readable meaning of those identifiers.**
Patient Data Accumulates Over Time	**The lecture uses a hypothetical patient to show how healthcare data develops across a timeline.** • the patient experiences: ○ an initial complaint of dysmenorrhea ○ missed work ○ medications ○ clinical visits ○ examinations ○ an ultrasound ○ diagnosis of an ovarian cyst ○ severe pain and hospitalization ○ diagnosis of endometriosis • each event creates additional data that can be stored in OMOP **These records may later support both patient care and research.**
Why Collect All This Data?	**Healthcare data supports two major purposes.** **CLINICAL CARE.** • historical data allows healthcare providers to understand:

- what has happened to the patient
- previous diagnoses
- previous treatments
- medications
- procedures
- other relevant medical history

- this information can improve future care

RESEARCH AND OPERATIONS.

- healthcare data can also support:
  - clinical research
  - hospital planning
  - staffing decisions
  - treatment research
  - analysis of diseases and outcomes
- historical de-identified records could be used to study the treatment of ovarian cysts or endometriosis

IMPORTANT DATABASE RELATIONSHIPS.

- a patient:
  - can have many visits
  - may interact with the healthcare system many times
  - can have many diagnoses
  - may develop many conditions over time

- one visit:
  - can have many diagnoses
  - can have many procedures
  - can have many measurements

- several procedures may occur
- many clinical measurements may be collected
- several conditions may be recorded

- codes :
  - are translated into meaningful clinical concepts using:
    - lookup tables
    - standardized vocabularies

As tables are connected, the amount of available data can grow rapidly.

OMOP Resources

The OMOP data model is managed by the OHDSI Consortium. OHDSI has a ton of great resources and online documentation that you may find useful, including:

- Book of OHDSI
  - part textbook on how OMOP works
  - part documentation of the OMOP data model
  - if you like to learn through reading, this is a great way to get started with using and understanding OMOP
- EHDEN Academy
  - offers a number of detailed video lectures covering how to use OMOP
  - you can practice many of the exercises from these lessons in our class database
- OMOP documentation
  - is a useful reference that defines each field across all OMOP tables
- ATHENA

- is an online tool where you can browse, search, and filter all the medical terminology concepts used in OMOP